# IcomRigControl — What This App Can Do (Capabilities Inventory)

*A complete, plain-language inventory of everything IcomRigControl does today, plus what's planned but not built. Meant as a shared reference and a brainstorming springboard. Grounded in the real code as of 2026-08-15 (Phases 1–10 complete, 283 passing tests).*

**In a nutshell:** IcomRigControl is a free, cross-platform (Windows/macOS/Linux/Raspberry Pi) control and logging program for the Icom IC-7300, IC-7300MK2, and IC-705 radios. It does everything Icom's own RS-BA1 remote software does *except* live remote audio — and adds a whole logging-and-operations ecosystem (QSO log, contests, callsign lookup, LoTW, HRD, N1MM/WSJT-X, APRS beaconing) that RS-BA1 has none of.

## 1. Radios & connection

| Capability | Detail |
|---|---|
| **Supported radios** | Icom IC-7300 (CI-V address `0x94`) and IC-7300MK2 (`0xB6`) |
| **Local connection** | USB / serial CI-V, selectable COM port / `/dev/tty…`, 115200 baud default |
| **Remote connection** | TCP to a networked `CivTcpServer` (Phase 9) — token-authenticated |
| **Demo mode** | Full UI with a simulated radio, no hardware needed |
| **IC-705** | Implemented (address `0xA4`); meter scaling shares the IC-7300 decoder, pending real-hardware verification |

## 2. Core radio control

- **Frequency** — read live from the radio and set it (BCD-encoded CI-V, verified against the reference).
- **Mode** — LSB, USB, AM, CW, RTTY, FM, CW-R, RTTY-R, DV — read and set.
- **PTT (transmit) control** — key the radio on/off from the app (CI-V `1C 00`), with the safety guarantees hardened in this session (never stuck on, never unidentified).
- **Remote power ON/OFF** — power the radio on/off from the app (CI-V `18 01` / `18 00`). Power-on sends a wake-up preamble and only works if the radio is in CI-V standby (matches RS-BA1's own limitation).
- **CW keyer & voice memories** — a Keyer window sends editable CW macros as Morse (CI-V `17`, up to 30 chars) and fires the radio's recorded voice memories T1–T8 (CI-V `28`).
- **FT8 one-touch setup** — an "FT8 Setup" button puts the radio in USB-D + wide filter via CI-V (`26 00 01 01 01`), replicating the radio's FT8 preset (which isn't itself CI-V-accessible). WSJT-X still does the decode; NB/NR/AGC are left manual, as Icom's own preset does.
- **VFO / split** — VFO select, A=B, swap, split control commands are implemented in the engine.
- **Live meters, polled ~continuously:** S-meter (S-units + true dBm), RF power output %, SWR, ALC, supply voltage (Vd), current draw (Id).

## 3. Spectrum scope & waterfall (Phase 7)

- Live **spectrum scope** capture from the radio (`27 xx` commands).
- **Waterfall display** with a real **frequency axis** (labels in kHz/MHz).
- **Click-to-tune** — click a point on the waterfall and the radio tunes there.
- Configurable span.

## 4. Memory channel editor (Phase 4)

- **Bulk read** all 99 memory channels from the radio (skips empty ones), with progress.
- **Bulk write** channels back to the radio.
- Add/edit channels in a grid (frequency, mode).
- Cancelable long reads.

## 5. QSO logging — the resilient backup of record (Phase 8)

*The core design principle: every contact lands in IcomRigControl's own local log, independent of any external program's health. Hardened this session so a QSO can never be lost to a write failure or a thread race.*

- **Log a QSO** — auto-fills frequency, band, and mode from the live radio at the moment of logging.
- **Write-through persistence** — every QSO is immediately written to a timestamped ADIF session file (crash-safe).
- **ADIF export** — standard `.adi` output.
- **General + contest logging** in one UI, with a contest selector.
- **Contests supported (verified against official ARRL rules):** ARRL Field Day and ARRL RTTY Roundup — including exchange fields, dupe checking, serial numbers, and live score.

## 6. Callsign lookup (Phase 8c)

- Look up a callsign to auto-fill name and grid square while logging.
- **Sources:** QRZ.com (XML), HamQTH (XML), Callook.info (US) — user-selectable.
- Lookups are contractually non-throwing (never crash the logger); hardened with an extra guard this session.

## 7. Logbook of the World / LoTW (Phase 8d)

- **Upload** logged QSOs to ARRL LoTW (signing delegated to ARRL's own TQSL tool — never reimplemented).
- **Download confirmations** and mark confirmed QSOs (✓) in the log grid.

## 8. Ham Radio Deluxe integration (Phase 8e)

- One-way, best-effort bridge that writes QSOs into HRD Logbook's SQLite database (defensive, schema-checked). HRD being down never blocks the local log.

## 9. N1MM Logger+ / WSJT-X integration (Phase 8f)

- **Send** live rig state (frequency, mode, PTT) out as RadioInfo UDP packets, to one or more user-configured destinations (127.0.0.1 or explicit LAN).
- **Receive** logged contacts over UDP from N1MM/WSJT-X/HRD and fold them into the local log.

## 10. Activity logging (Phase 5)

- Continuous **CSV activity log** of rig state over time (timestamp, frequency, mode, all meter readings) for later analysis. Separate from the QSO log.

## 11. Remote / network operation (Phase 9)

- **Headless server mode** (`IcomRigControl.UI --headless-server`) — runs with no display, serves CI-V control to remote clients over TCP. Designed for a Raspberry Pi sitting next to the radio, reachable over LAN, VPN, or 44Net/AMPRNet.
- **Token authentication** required (never runs with a blank token).
- The desktop app can act as a **remote client** — from the app's perspective a remote radio is indistinguishable from a local one; every feature above works remotely.
- *Proven via loopback; real-hardware-over-real-network test still pending.*

## 12. APRS beacon over HF (Phase 10)

*Neither IC-7300 has a TNC — this is built as software AFSK/AX.25 packet audio, complete and confirmed audible on both Windows and macOS.*

- Build an **APRS position beacon** (lat/lon, symbol, comment), auto-appending live frequency/mode.
- **AX.25 UI frame** construction + **AFSK modulation** to audio (300-baud HF profile).
- Keys PTT, plays the packet audio through a chosen sound device, releases PTT — with the safety fixes from this session (guaranteed key-down, no cross-keying, callsign validated before transmit).
- **Manual** "Send Beacon" and **automatic** periodic beaconing at a configured interval.

## 13. Settings & platform

- One Settings window covering connection, APRS, callsign lookup, LoTW/TQSL, HRD, and N1MM.
- Cross-platform audio (NAudio/WASAPI on Windows, `afplay` on macOS).
- Settings persisted to JSON; secrets kept out of source control.

- **Windows, macOS, Linux desktop, and Raspberry Pi OS** all supported via Avalonia.

## 14. Architecture (for context)

Four clean layers: **CivEngine** (CI-V framing, serial I/O, APRS/AFSK) → **RigModel** (`Transceiver` + the network layer) → **Services** (all the integrations above) → **UI** (Avalonia views/view-models). 283 automated tests.

## What it does NOT do yet — the brainstorm springboard

These are the natural places to take it next. Items marked *(documented)* already have notes in CLAUDE.md.

**Small / near-term**

- ~~Remote Power ON/OFF~~ — **DONE** (implemented 2026-08-15; CI-V `0x18`, Power On/Off buttons on the dashboard).
- **Real-hardware validation** of Phase 9 remote mode over an actual network link.
- **Meter scaling verification** — Po/ALC/scope-span byte layouts need confirming against a real radio (flagged in the audit).

**Medium**

- **IC-705 real-hardware verification** — support is now implemented (first pass: enum + address `0xA4` + Settings picker). The remaining step is confirming meter scaling byte-for-byte against the real radio. (GPS/D-PRS and D-STAR remain out of scope.)
- **UI redesign** *(documented)* — you've said the current UI isn't satisfying; this is the gate for the comprehensive User Manual. Open questions: color/theme, transceiver-panel vs. flat dashboard, tabs vs. one scrolling window.
- **Phase 11: clickable radio front-panel** *(documented)* — a photo/vector of the rig with clickable controls.

**Large / its own project**

- **Phase 12: Remote Audio** *(documented)* — real-time low-latency audio capture + streaming + playback over IP, to fully match RS-BA1. Deliberately scoped as its own multi-session phase (needs a new capture interface, streaming protocol, jitter buffering).
- **Comprehensive User Manual** *(documented)* — real screenshots, click-by-click, every quirk; triggered once the UI-redesign question is resolved.

**Decided / in progress (2026-08-15)**

- ~~CW keyer / voice memories~~ — **DONE** (Keyer window; CI-V 17 / 28).
- **DX Cluster + band map** — planned next: telnet to a cluster, show spots on the waterfall, click-to-tune.
- **Rotator / antenna switch / amplifier control** — **not planned**: not CI-V functions, and no test hardware on hand (would ship unverifiable support).

- **Digital modes** — FT8 stays **WSJT-X only** (no in-app FT8 — reimplementing it isn't sane); an **"FT8 Setup" button** now replicates the radio's FT8 preset (USB-D + wide filter) over CI-V as a convenience. In-app **RTTY and CW decode** are feasible "path 2" features but are **gated on Phase 12 audio capture**, so deferred until after the audio work. WSJT-X integration is untouched.
- More contests; general-logging niceties (QSL cards, statistics, maps).
- Mobile/tablet or web front-end to the headless server.